

Coordinate Frames — and how a waypoint went 90° wrong

Gazebo, ArduPilot and ROS each describe the world differently. Reading a position out of one and giving it to another without converting is the single easiest way to send a drone somewhere it was never meant to go.

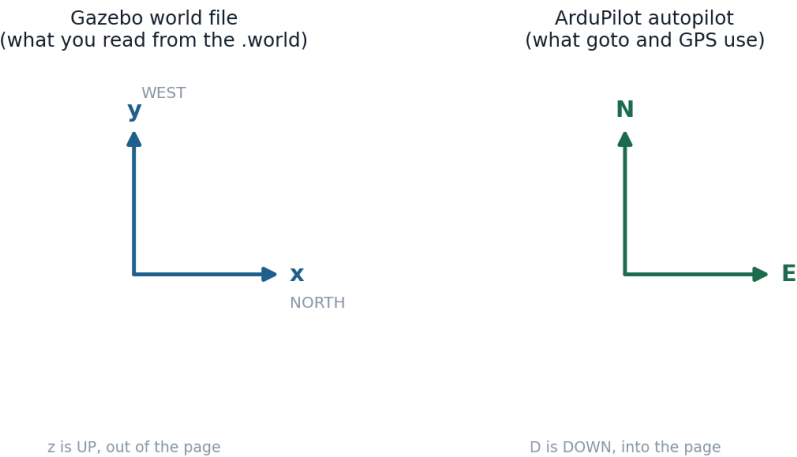
1. Three frames, three conventions

A **frame** is just an agreement about which way x, y and z point. Nothing is more correct than anything else — but two programs must agree, or the numbers mean different things.

Used by	Convention	x	y	z
ROS 2, Gazebo (default)	ENU	East	North	Up
ArduPilot, PX4, aviation	NED	North	East	Down
This project's world	see below	North	West	Up

ENU is the ROS standard, fixed by **REP-103**. NED is the aviation standard. Both are everywhere; neither is going away.

! The trap: both use the letters x and y. A file saying $x = 40.5$ tells you nothing until you know which convention it follows. That single ambiguity caused every failed flight in this project until 2026-08-30.



$$\text{north} = \text{gazebo_x} \qquad \text{east} = - \text{gazebo_y}$$

set by `<gazeboXYZToNED>0 0 0 3.141593 0 0</gazeboXYZToNED>` in the model SDF

Figure 1 — the world file's axes (left) and what the autopilot actually receives (right). The link between them is set by one line of the model SDF.

2. What connects them

The ArduPilot Gazebo plugin is told how to convert, in the model SDF:

```
<plugin name="arducopter_plugin" filename="libArduPilotPlugin.so">
  <gazeboXYZToNED>0 0 0 3.141593 0 0</gazeboXYZToNED>
```

Those six numbers are x y z roll pitch yaw. Here it is a roll of π radians — 180° about the x axis — which flips the sign of y and z:

```
north = gazebo_x
east  = -gazebo_y
down  = -gazebo_z
```

So in this project Gazebo's x is **north** and its y is **west**, not the ENU default. That is a property of this world and this plugin setting, not of Gazebo in general — which is exactly why it must be measured rather than assumed.

3. The error, in one picture

The mission needed to fly from the drone's spawn point to five people standing in a city square. Both positions were read from the same `.world` file:

```
<model name="iris_1_demo">   <pose>-70   -22   0   0   0   0</pose>
<model name="person_center"> <pose> 40.5 -187.9 0   0   0 1.57</pose>
```

Subtracting gives +110.5 and -165.9. The mistake was assuming those were *east* and *north*. They are *north* and *west*, so the correct autopilot offsets are +110.5 north and +165.9 east — the two numbers swapped, and one sign flipped.

Same two numbers, axes swapped — 90° off

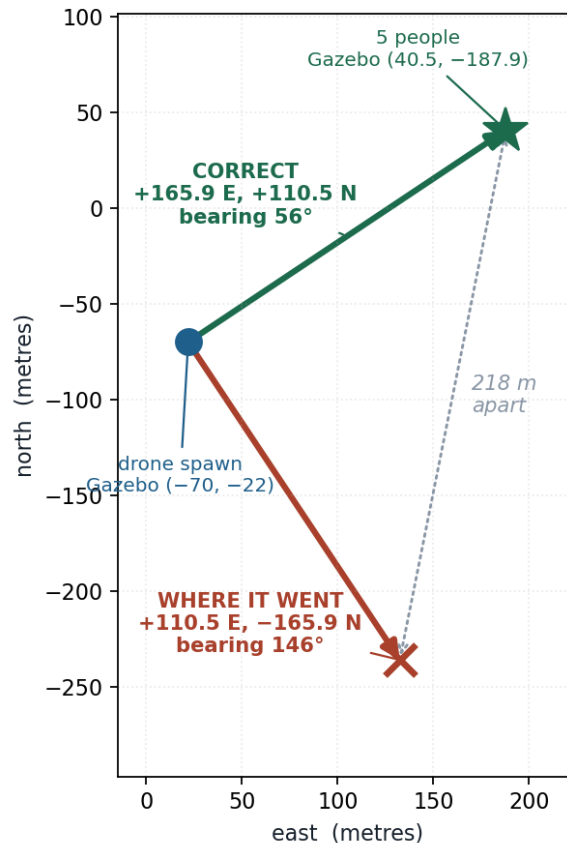


Figure 2 — same distance, wrong direction. The drone flew 199 m on a bearing of 146° instead of 56°, ending 218 m from anybody. Every detection run before the fix searched empty ground.

! The failure was silent. The drone armed, climbed, flew a sensible distance, held station and returned home. Every log line said success. Only the detector's *0 humans*, run after run, showed anything was wrong — and that looked like a camera problem, not a navigation one.

4. World frame or drone frame? Both — for different things

This simulation uses **both**, and knowing which applies where is what makes the geometry predictable.

What	Frame	Why
goto waypoints	WORLD (global lat/lon)	MAV_FRAME_GLOBAL_RELATIVE_ALT_INT — an absolute place on Earth. Heading has no effect on where the drone ends up.
GPS telemetry	WORLD (global lat/lon)	what /uavN/gps reports
ns-3 node positions	WORLD (Gazebo metres)	/uav_world_positions feeds the channel model in Gazebo coordinates
Camera field of view	BODY (moves with the drone)	the sensor is bolted to the airframe, so what it sees depends on where the aircraft is pointing
FDM / IMU	BODY	accelerations and rates are felt in the airframe

goto uses the WORLD frame · the camera uses the BODY frame

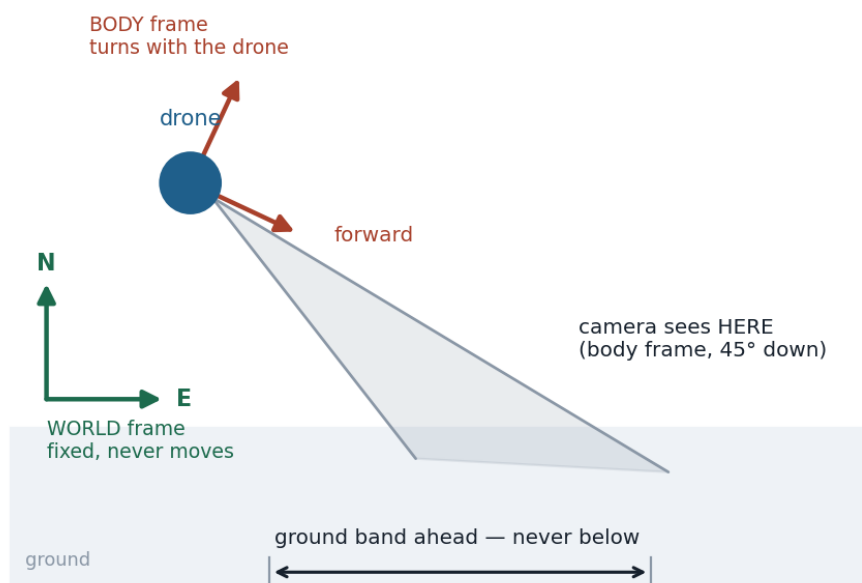


Figure 3 — a waypoint is a fixed point on the ground; the camera's view is a cone attached to the aircraft. Turn the drone and the waypoint does not move, but everything the camera sees does.

5. Why the camera never sees straight down

The camera is pitched 45° **forward**-down, so at altitude A it sees the ground roughly 0.7A to 1.5A metres ahead. Directly beneath the aircraft is a blind spot. A drone hovering exactly over its subjects cannot see them.

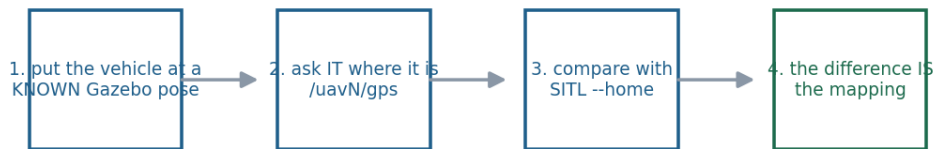
Altitude	Ground band ahead	Stand off by
25 m	17 – 36 m	25 m
15 m	10 – 22 m	15 m
10 m	7 – 15 m	10 m

Keep the stand-off roughly equal to the altitude. The 45° pitch is deliberate: a person's pixel height scales as $0.5 \cdot \sin(2 \cdot \text{pitch})$, which peaks exactly there; at nadir a standing person has no vertical extent.

6. How to get it right: ask, don't derive

The mapping was eventually found not by reasoning about conventions but by putting the vehicle at a known place and reading back what it thought its position was.

Ask the vehicle. Do not derive it.



spawned at Gazebo (-70, -22) → reported 70 m SOUTH, 22 m EAST
so north = gazebo_x and east = - gazebo_y

Figure 4 — four steps, no arithmetic. The vehicle is the authority on its own frame.

```
# 1. the world file says where it spawns
grep -A1 'iris_1_demo' worlds/*.world      -> pose -70 -22 0

# 2. ask the autopilot where it thinks it is
ros2 topic echo /uav1/gps --once           -> 6.078440, 80.191727

# 3. compare with SITL --home               6.079068, 80.191528
# difference: 70 m SOUTH, 22 m EAST

# 4. therefore north = gazebo_x east = -gazebo_y
```

! A derivation can be self-consistent and still wrong. An earlier check using the world's <spherical_coordinates> agreed to 0.14 m — but it converted both sides the same incorrect way, so it confirmed nothing. One measurement from the vehicle settled it immediately.

7. Where to read more

Source	What it gives you
REP-103 — Standard Units and Coordinate Conventions ros.org/reps/rep-0103.html	The ROS rule: ENU for the world, x-forward/y-left/z-up for a body. The document everything ROS is measured against.
REP-105 — Coordinate Frames for Mobile Platforms ros.org/reps/rep-0105.html	map, odom and base_link, and how they relate.
ArduPilot — SITL with Gazebo ardupilot.org/dev/docs/sitl-with-gazebo.html	The plugin, its SDF parameters, and the frame conversion it performs.
MAVLink — MAV_FRAME mavlink.io/en/messages/common.html#MAV_FRAME	Every frame a command can be expressed in. GLOBAL_RELATIVE_ALT_INT is the one this project's goto uses.
Gazebo Classic — model SDF sdformat.org/spec	What a <pose> means: six numbers, x y z roll pitch yaw, relative to the parent frame.

OK Rules of thumb worth keeping: (1) a pose is meaningless without its frame; (2) prefer relative offsets over absolute coordinates, because a frame error cancels; (3) when two systems disagree, believe the one that is flying.